Python - Access Dictionary Items

Access Dictionary Items

Accessing dictionary items in Python involves retrieving the values associated with specific keys within a dictionary data structure. Dictionaries are composed of key-value pairs, where each key is unique and maps to a corresponding value. Accessing dictionary items allows you to retrieve these values by providing the respective keys.

There are various ways to access dictionary items in Python. They include −

- Using square brackets []
- The get() method
- Iterating through the dictionary using loops
- Or specific methods like keys(), values(), and items()

We will discuss each method in detail to understand how to access and retrieve data from dictionaries.

Access Dictionary Items Using Square Brackets []

In Python, the square brackets **[]** are used for creating lists, accessing elements from a list or other iterable objects (like strings, tuples, or dictionaries), and for list comprehension.

We can access dictionary items using square brackets by providing the key inside the brackets. This retrieves the value associated with the specified key.

Example 1

In the following example, we are defining a dictionary named "capitals" where each key represents a state and its corresponding value represents the capital city.

Then, we access and retrieve the capital cities of Gujarat and Karnataka using their respective keys 'Gujarat' and 'Karnataka' from the dictionary –

```
capitals = {"Maharashtra":"Mumbai", "Gujarat":"Gandhinagar",
"Telangana":"Hyderabad", "Karnataka":"Bengaluru"}
print ("Capital of Gujarat is : ", capitals['Gujarat'])
print ("Capital of Karnataka is : ", capitals['Karnataka'])
```

It will produce the following output –

```
Capital of Gujarat is: Gandhinagar
Capital of Karnataka is: Bengaluru
```

Example 2

Python raises a **KeyError** if the key given inside the square brackets is not present in the dictionary object –

```
capitals = {"Maharashtra":"Mumbai", "Gujarat":"Gandhinagar",
"Telangana":"Hyderabad", "Karnataka":"Bengaluru"}
print ("Captial of Haryana is : ", capitals['Haryana'])
```

Following is the error obtained –

```
   print ("Captial of Haryana is : ", capitals['Haryana'])
                                      ~~~~~~~~^^^^^^^^^^^^

KeyError: 'Haryana'
```

Access Dictionary Items Using get() Method

The get() method in Python's dict class is used to retrieve the value associated with a specified key. If the key is not found in the dictionary, it returns a default value (usually None) instead of raising a KeyError.

We can access dictionary items using the get() method by specifying the key as an argument. If the key exists in the dictionary, the method returns the associated value; otherwise, it returns a default value, which is often None unless specified otherwise.

Syntax

Following is the syntax of the get() method in Python –

```
Val = dict.get("key")
```

where, **key** is an immutable object used as key in the dictionary object.

Example 1

In the example below, we are defining a dictionary named "capitals" where each key-value pair maps a state to its capital city. Then, we use the get() method to retrieve the capital cities of "Gujarat" and "Karnataka" –

```
capitals = {"Maharashtra":"Mumbai", "Gujarat":"Gandhinagar",
"Telangana":"Hyderabad", "Karnataka":"Bengaluru"}
print ("Capital of Gujarat is: ", capitals.get('Gujarat'))
print ("Capital of Karnataka is: ", capitals.get('Karnataka'))
```

We get the output as shown below –

```
Capital of Gujarat is: Gandhinagar
Capital of Karnataka is: Bengaluru
```

Example 2

Unlike the "[]" operator, the get() method doesn't raise error if the key is not found; it return None –

```
capitals = {"Maharashtra":"Mumbai", "Gujarat":"Gandhinagar",
"Telangana":"Hyderabad", "Karnataka":"Bengaluru"}
```

```
print ("Capital of Haryana is : ", capitals.get('Haryana'))
```

It will produce the following output −

```
Capital of Haryana is : None
```

Example 3

The get() method accepts an optional string argument. If it is given, and if the key is not found, this string becomes the return value −

```
capitals = {"Maharashtra":"Mumbai", "Gujarat":"Gandhinagar",
"Telangana":"Hyderabad", "Karnataka":"Bengaluru"}
print ("Capital of Haryana is : ", capitals.get('Haryana', 'Not found'))
```

After executing the above code, we get the following output −

```
Capital of Haryana is: Not found
```

Access Dictionary Keys

In a dictionary, keys are the unique identifiers associated with each value. They act as labels or indices that allow you to access and retrieve the corresponding value. Keys are immutable, meaning they cannot be changed once they are assigned. They must be of an immutable data type, such as strings, numbers, or tuples.

We can access dictionary keys in Python using the keys() method, which returns a view object containing all the keys in the dictionary.

Example

In this example, we are retrieving all the keys from the dictionary "student_info" using the keys() method −

```
```

```
# Creating a dictionary with keys and values
student_info = {
    "name": "Alice",
    "age": 21,
    "major": "Computer Science"
}
# Accessing all keys using the keys() method
all_keys = student_info.keys()
print("Keys:", all_keys)
```

Following is the output of the above code −

```
Keys: dict_keys(['name', 'age', 'major'])
```

Access Dictionary Values

In a dictionary, values are the data associated with each unique key. They represent the actual information stored in the dictionary and can be of any data type, such as strings, integers, lists, other dictionaries, and more. Each key in a dictionary maps to a specific value, forming a key-value pair.

We can access dictionary values in Python using −

- **Square Brackets ([]) −** By providing the key inside the brackets.
- **The get() Method −** By calling the method with the key as an argument, optionally providing a default value.
- **The values() Method −** which returns a view object containing all the values in the dictionary

Example 1

In this example, we are directly accessing associated with the key "name" and "age" using the sqaure brackets −

```
# Creating a dictionary with student information
student_info = {
    "name": "Alice",
```

```
    "age": 21,
    "major": "Computer Science"
}
# Accessing dictionary values using square brackets
name = student_info["name"]
age = student_info["age"]
print("Name:", name)
print("Age:", age)
```

Output of the above code is as follows –

```
Name: Alice
Age: 21
```

Example 2

In here, we use the get() method to retrieve the value associated with the key "major" and provide a default value of "2023" for the key "graduation_year" –

```
# Creating a dictionary with student information
student_info = {
    "name": "Alice",
    "age": 21,
    "major": "Computer Science"
}
# Accessing dictionary values using the get() method
major = student_info.get("major")
# Default value provided if key is not found
grad_year = student_info.get("graduation_year", "2023")

print("Major:", major)
print("Graduation Year:", grad_year)
```

We get the result as follows –

```
Major: Computer Science
Graduation Year: 2023
```

Example 3

Now, we are retrieving all the values from the dictionary "student_info" using the values() method –

```
# Creating a dictionary with keys and values
student_info = {
    "name": "Alice",
    "age": 21,
    "major": "Computer Science"
}
# Accessing all values using the values() method
all_values = student_info.values()
print("Values:", all_values)
```

The result obtained is as shown below –

```
Values: dict_values(['Alice', 21, 'Computer Science'])
```

Access Dictionary Items Using the items() Function

The items() function in Python is used to return a view object that displays a list of a dictionary's key-value tuple pairs.

This view object can be used to iterate over the dictionary's keys and values simultaneously, making it easy to access both the keys and the values in a single loop.

Example

In the following example, we are using the items() function to retrieve all the key-value pairs from the dictionary "student_info" –

```
# Creating a dictionary with student information
student_info = {
    "name": "Alice",
    "age": 21,
```

```
    "major": "Computer Science"
}

# Using the items() method to get key-value pairs
all_items = student_info.items()
print("Items:", all_items)
# Iterating through the key-value pairs
print("Iterating through key-value pairs:")
for key, value in all_items:
    print(f"{key}: {value}")
```

Following is the output of the above code −

```
Items: dict_items([('name', 'Alice'), ('age', 21), ('major', 'Computer Science')]
Iterating through key-value pairs:
name: Alice
age: 21
major: Computer Science
```

Microsoft Azure Tutorial

Git Tutorial

Ethical Hacking Tutorial

Docker Tutorial

Kubernetes Tutorial

DSA Tutorial

Spring Boot Tutorial

SDLC Tutorial

Unix Tutorial

CERTIFICATIONS

Business Analytics Certification

Java & Spring Boot Advanced Certification

Data Science Advanced Certification

Cloud Computing And DevOps

Advanced Certification In Business Analytics

Artificial Intelligence And Machine Learning

DevOps Certification

Game Development Certification

Front-End Developer Certification

AWS Certification Training

Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler

Online Python Compiler

Online Go Compiler

Online C Compiler

Online C++ Compiler

Online C# Compiler

Online PHP Compiler

Online MATLAB Compiler

Online Bash Terminal

Online SQL Compiler

Online Html Editor